



Universidad Nacional de Colombia
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas e Industrial
2025

AWS Lambda – Análisis

Jenny Catherine Herrera Garzon - jcherreraga@unal.edu.co

1. ¿Qué es AWS Lambda?

AWS Lambda es un servicio de **ejecución de funciones bajo demanda** dentro de la nube de Amazon.

En lugar de mantener un servidor encendido, se sube código y se configura para que este se ejecute únicamente cuando ocurre un evento.

La idea central es:

- No gestionar servidores ni sistemas operativos.
- Ejecutar pequeñas piezas de lógica (funciones) cuando hace falta.
Pagar solo por el tiempo de ejecución y las invocaciones.

2. Cómo funciona, a nivel general

El flujo típico de uso de Lambda puede resumirse así:

1. Se escribe una función (por ejemplo en Node.js o Python).
2. Se despliega en AWS Lambda (subiendo un zip, usando una herramienta de infraestructura como código, etc.).

3. Se configura un **disparador** (trigger), es decir, un servicio de AWS que enviará eventos:
 - API Gateway para peticiones HTTP/REST.
 - S3 para reaccionar a la subida de archivos.
 - SQS para procesar mensajes de una cola.
 - EventBridge/CloudWatch Events para tareas programadas (tipo cron).
4. Cuando ocurre el evento, Lambda:
 - Crea o reutiliza un entorno de ejecución.
 - Ejecuta la función con los datos del evento.
 - Libera el entorno cuando ya no es necesario.
5. El cobro se hace en función de:
 - Número de invocaciones.
 - Tiempo de ejecución y memoria asignada.

3. Conceptos clave

Al trabajar con Lambda es útil tener claros algunos términos:

- **Función Lambda**
Unidad básica de trabajo: código + configuración (runtime, memoria, timeout, variables de entorno, permisos).
- **Evento**
Datos que llegan a la función cuando algo la dispara.
Ejemplos: cuerpo de una petición HTTP, metadatos de un archivo S3, contenido de un mensaje en SQS.
- **Trigger (disparador)**
Servicio que genera el evento: API Gateway, S3, SQS, EventBridge, entre otros.
- **Runtime**
Lenguaje y versión con la que se ejecuta el código (por ejemplo, Node.js 20, Python 3.12).

- **Timeout**

Tiempo máximo que puede durar una invocación antes de que Lambda la corte.

- **Cold start**

Primera ejecución de una función (o ejecución después de un período de inactividad).

Lambda necesita preparar el entorno y esto introduce una pequeña latencia adicional.

Cuando el entorno ya está “caliente”, las siguientes invocaciones suelen ser más rápidas.

4. Cuándo tiene sentido usar Lambda

AWS Lambda resulta especialmente útil en escenarios donde se necesita:

- **Procesamiento por eventos**

- Reaccionar a la subida de archivos (por ejemplo, imágenes en S3).
- Procesar mensajes que llegan por una cola o un topic.
Ejecutar lógica cada vez que ocurre una acción específica en el sistema.

- **APIs ligeras o microservicios simples**

- Endpoints REST con lógica acotada.
- Servicios pequeños que consultan una base de datos, validan datos y devuelven una respuesta.

- **Automatizaciones y tareas programadas**

- Limpieza periódica de datos.
- Generación de reportes.
- Tareas de mantenimiento que no requieren un servidor dedicado.

En general, Lambda es adecuado para **funciones cortas, independientes y sin estado**, que se beneficien de ejecutarse solo cuando realmente se necesitan.+

5. Cuándo no es la mejor opción

Conviene considerar otras alternativas (EC2, contenedores ECS/EKS, etc.) cuando:

- Se requiere un proceso **corriendo de forma continua**:
 - Conexiones persistentes (por ejemplo, websockets de larga duración).
 - Servidores de juegos o servicios en tiempo real que no pueden apagarse entre peticiones.
- Se manejan **cargas muy pesadas o de larga duración**:
 - Procesos intensivos que consumen mucho CPU o duran varios minutos de forma constante.
- Se necesita **control detallado del sistema operativo**:
 - Instalación de software muy específico.
 - Uso de drivers o configuraciones que no encajan bien en el modelo de Lambda.
- La aplicación depende de **estado en memoria** entre invocaciones:
 - Lambda está pensado para ser *stateless*.
El estado debe almacenarse en bases de datos, caches o colas, no en la memoria de la función.

6. Ventajas principales

Algunas ventajas prácticas de Lambda:

- No requiere administrar infraestructura de servidores.
- Escala automáticamente según la cantidad de eventos.
- El cobro está ligado al uso real (invocaciones y tiempo de ejecución).
- Integración directa con servicios nativos de AWS (S3, DynamoDB, SQS, SNS, API Gateway, entre otros).
- Facilita arquitecturas modulares, donde cada función resuelve una tarea específica.

7. Limitaciones y puntos a vigilar

Aunque es muy útil, también presenta restricciones y detalles a tener en cuenta:

- **Cold starts**
Pueden introducir latencia adicional en ciertos casos, especialmente si se usan runtimes pesados o funciones poco invocadas.
- **Límites de tiempo y recursos**
Cada función tiene un tiempo máximo de ejecución y límites de memoria y tamaño del paquete de despliegue.
- **Depuración y observabilidad**
El flujo de trabajo de debugging no es idéntico al de un servidor tradicional. Suelen usarse logs (por ejemplo, en CloudWatch) y herramientas de monitoreo para entender qué sucede.
- **Diseño de funciones**
Si se concentra demasiada lógica en una sola función, se pierde claridad y se hace más difícil mantener el sistema.
Es recomendable mantener funciones pequeñas y con responsabilidades bien definidas.

8. Ejemplo conceptual

Un ejemplo típico que ilustra el uso de Lambda:

1. Una persona sube una imagen a un bucket de S3.
2. Ese evento dispara una función Lambda.
3. La función:
 - Lee la imagen original.
 - Genera una versión en miniatura.
 - Guarda la miniatura en otro bucket S3.
4. La ejecución termina y el entorno de Lambda se libera.

Este flujo permite procesar imágenes de forma automática y escalable, sin mantener un servidor dedicado para ello.